MOD -03 - JDK new Features

MOD -03 - JDK new Features

LUMINAR
TECHNOLAB

Stream API

The new java.util.stream package has been added to JDK 8 which helps java developers to perform operations like search, filter, map, reduce, or manipulate collections.

Assume that :

- ✓ Stream as a mechanism to create a sequence of elements supporting sequential and parallel aggregate operations.
- ✓ Ie. we can anytime collect and perform some operation of all elements present in the stream in one call.

Stream.filter()

The filter() method is an intermediate operation of the Stream interface that allows us to filter elements of a stream that match a given Predicate:

we can use stream and predicate to –

1. first filter certain elements from a group, and
2. then perform some operation on filtered elements.

Predicate

In mathematics, a predicate is commonly understood to be a boolean-valued function

P:X ? {true, false}

called the predicate on X

- ✓ Introduced in Java 8, Predicate is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.
- ✓ Java Predicate interface helps in writing filter expressions so easy.
- ✓ we can use these true/false returning functions in day-to-day programming.

Ie. we can use predicates anywhere we need to evaluate a condition on a collection of objects such that evaluation can result either in true or false.

```
@FuncionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

E.g. we can use predicates in these real-life usecases/scenarios:

1. Find all children born after a particular date
2. Pizzas ordered within a specific time range
3. Employees older than certain age

Predicate with Stream

Predicate is a functional interface that means we can pass it in lambda expressions wherever a predicate is expected.

For example, one such method is filter() method from Stream interface.

Lambda Expression and Stream API

```java
package streampackage;

import java.util.Arrays;

public class StreamSimpleProgram {
    public static void main(String[] args) {

        List<String> names = Arrays.asList("BiniBindu","Rukku","Nitya","Ani");

        //Creating the stream of all names
        Stream<String> allNames = names.stream();

        //Creating another stream by filtering length above 6 using filter()
        Stream<String> longNames = allNames.filter(str -> str.length() > 6);


        //displaying the long names using forEach
                longNames.forEach(str->System.out.print(str+" "));

```

```java

        //method chaining

        List filteredList = names.stream() // converting the list to stream
                        .filter(s -> s.length() == 5) // filter the stream to create a new stream
                        .filter(s -> !s.startsWith("d"))
                        .filter(s -> s.contains("R"))
                        .collect(Collectors.toList()) ;// collect the final stream and convert it to a List

            System.out.println("\n"+filteredList);
            //or

            filteredList.forEach(System.out::println);
      }

}

```

Scenario: we have a stream of data (e.g.List of String) where each string is a combination of country name and tourists' places of the country. We want to retrieve only the places from India

Note:

- For this we can use methods like filter(), map() and forEach() of the Stream API
- These methods can take a lambda expression as input.
- Java stream provides a method filter() to filter stream elements on the basis of given predicate(p)
- Stream map(Function mapper) returns a stream consisting of the results of applying the given function to the elements of this stream.
- sort a stream of strings using Stream.sorted() method

```java
package functionalinterfacelambda;

import java.util.ArrayList;
import java.util.List;

public class LambdaStream {

    // method for preparing our data
    List<String> getPlaces(){
        // create an object of list using ArrayList
        List<String> places = new ArrayList<>();

        // add places and country to the list
        places.add("Australia, Melbourne");
        places.add("Australia, Sydney");
        places.add("India, Kerala");
        places.add("India, Tamil Nadu");
        places.add("India, Delhi");
        places.add("USA, New York");

        return places;
    }
```

```java
    public static void main( String[] args ) {

        LambdaStream ls=new LambdaStream();

        List<String> myPlaces = ls.getPlaces();
        System.out.println("Places from India:\n========================");

        // Filter places from India
        myPlaces.stream()
                .filter((p) -> p.startsWith("India"))
                .map((p) -> p.toUpperCase())
                .sorted()
                .forEach((p) -> System.out.println(p));
    }

}
```

Practice Programs

Scenario: Before and After JDK 8

```java
package streampackage;

public class Student {

    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

```

```java
package streampackage;


import java.util.Arrays;
import java.util.List;

public class BeforeStream {

    private static Student getStudentByName(List<Student> students, String name) {

        Student result = null;
        for (Student temp : students) {
            if (name.equals(temp.getName())) {
                result = temp;
            }
        }
        return result;
    }
```

```java

    public static void main(String[] args) {

        List<Student> students = Arrays.asList(
                new Student("Bini", 30),
                new Student("Bindu", 20),
                new Student("Nitya", 40)
        );

        Student result = getStudentByName(students, "Bini");
        System.out.println(result.getName()+""+result.getAge());

    }


}

```

```java
package streampackage;

import java.util.Arrays;
import java.util.List;

public class AfterStream {
    public static void main(String[] args) {

        List<Student> students = Arrays.asList(
                new Student("Bini", 30),
                new Student("Bindu", 20),
                new Student("Nitya", 40)
        );

        Student firstResult = students.stream()                    // Convert to steam
                .filter(x -> "Bini".equals(x.getName()))        // we want "Bini" only
                .findAny()                                      // If 'findAny' then return found
                .orElse(null);                                  // If not found, return null

        System.out.println(firstResult.getName()+""+firstResult.getAge());
    }
}
```

Practice Programs

```java
package streamapipack;


import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.stream.Collectors;


public class StreamPrograms {


public static void main(String[] args) {
System.out.println("Q1: Find even numbers in a list without using conditional
statements\n==============================");
List<Integer> numbers=Arrays.asList(23,24,56,78,35,82,90);
//with conditional statements
//for(Integer i:numbers)
//if(i%2==0)
//System.out.println(i);
```

```java
//using stream api -> without conditional statements
numbers.stream()
.filter(i->i%2==0)
.forEach(System.out::println);


System.out.println("\nQ2 . Find duplicates\n==============================");
Set<Integer> seen=new HashSet<>();
numbers.stream()
.filter(p->!seen.add(p))
.forEach(System.out::println);


System.out.println("\nQ3 . Sort a list\n==============================");
numbers.stream()
.sorted()
.forEach(System.out::println);
```

```java
System.out.println("\nQ4 . Find max number\n==================================");
int maxNumber=numbers.stream()
.max((m,n)->m.compareTo(n))
.get();
System.out.println("max number="+maxNumber);


//or
Optional<Integer> max = numbers.stream()
.max(Integer::compare);
System.out.println("max number="+max.get());
```

```java
System.out.println("\nQ5 . Convert list to Map\n==================================");

Map<Integer, String> map = numbers.stream()
.collect(Collectors.toMap(//collect() transforms the stream into a different
structure (here => Map)
n -> n, //key Mapper -> for each element in the list it returns n itself as key
n -> "Number " + n //value Mapper -> For each element n It creates a String value
"Number"+n's value
));

System.out.println("\n print map using for each\n==================================");
map.forEach((key, value) -> {
System.out.println(key + " -> " + value);
});

System.out.println("\n print map using for Map.Entry\n=====================");
for (Map.Entry<Integer, String> entry : map.entrySet()) {
System.out.println(entry.getKey() + " -> " + entry.getValue());
}
}


}
```

Practice Programs

Find the second highest salary from a list of employees

```java
package streamapipack;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

class Employee {
String name;
double salary;

Employee(String name, double salary) {
this.name = name;
this.salary = salary;
}
public double getSalary() {
return salary;
}

public String getName() {
return name;
}
}
```

```java
public class StreamEmployeeSecondHighestSalary {
public static void main(String[] args) {

List<Employee> employees = Arrays.asList(
new Employee("Bini", 50000),
new Employee("Bindu",75000),
new Employee("Cini", 60000),
new Employee("Devu", 70000)
);

Optional<Double> secondHighest = employees.stream()
.map(Employee::getSalary) // extract salary
.distinct() // remove duplicates
.sorted(Comparator.reverseOrder()) // sort descending
.skip(1) // skip highest
.findFirst(); // get second highest

if (secondHighest.isPresent()) {
System.out.println("Second Highest Salary: " + secondHighest.get());
} else {
System.out.println("Not enough data");
}
```

```
System.out.println("Highest salaried Employee \n============================");
employees.stream()
.map(Employee::getSalary)
.sorted(Comparator.reverseOrder())
.skip(0)
.findFirst()
.ifPresent(System.out::println);//.ifPresent(e -> System.out.println(e.getName()
+ " -> " + e.getSalary()));
}


}
```

Code explanation

```
Optional<Double> secondHighest =
```

Optional is a container that may or may not contain a value
it may contain:

- Second highest salary ✅
- OR nothing ❌

```
employees.stream()
```

Converts the List<Employee> into a stream
Stream = sequence of elements for processing

```
.map(Employee::getSalary)
```

Employee::getSalary=> this is a method reference.
It means:“For each Employee, call getSalary() and return the result”
is equivalent to: .map(e -> e.getSalary())

```
.distinct()
```

Removes duplicate salaries

```
.sorted(Comparator.reverseOrder())
```

built-in comparator used to sort elements in descending order.
Sorts salaries in descending order

Employee("Bini", 50000) => 50000
Employee("Bindu", 75000) => 75000
Employee("Cini", 60000) => 60000

Use .map() when:
You want to extract a field
Convert object => value

`.skip(1)`
Skips the first element (highest salary)

`.findFirst();`
Picks the first element from remaining list

`secondHighest.ifPresentOrElse(`
Checks:If value exists → run first block
If empty → run second block

```
sal -> System.out.println("Second Highest Salary: " + sal),
```
sal = value inside OptionalPrints the result

```
() -> System.out.println("Not enough data")
```
Runs when no second highest exists(e.g., only 1 employee)

Tip:
What if only 1 employee?
Optional will be empty => "Not enough data" prints

Thank you . Happy Learning ☺